NKT
Tag 3: Listen, Such/Sortieralgorithmen, HOF

Tobias Wrede, Alexa Katharina-Ronja Brockmöller

11. März 2026

Gliederung

1. Listen
2. Binäre Suche
3. Sortieralgorithmen
4. Generics
5. Higher Order Functions und Streams

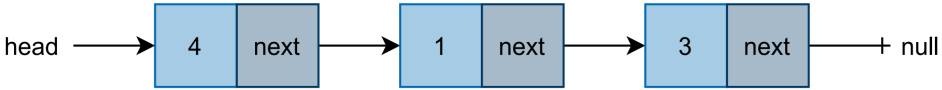
Gliederung

1. Listen
2. Binäre Suche
3. Sortieralgorithmen
4. Generics
5. Higher Order Functions und Streams

Listen

Warum Listen?

- erste Node wird *head* genannt (letzte wird *tail* genannt)
- jede Node hat einen Zeiger auf nächste Node
- sie sind dynamisch vergrößerbar
- effizient, wenn wir einfügen oder löschen wollen



Listentypen

- **Einfach verkettete Liste:** Jede Node zeigt nur auf die nächste Node.
- **Doppelt verkettete Liste:** Jede Node zeigt auf vorherige und nächste Node.

Node-Klasse

```
class LinkedList {
    private Node head;

    private class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
    (...)
}
```

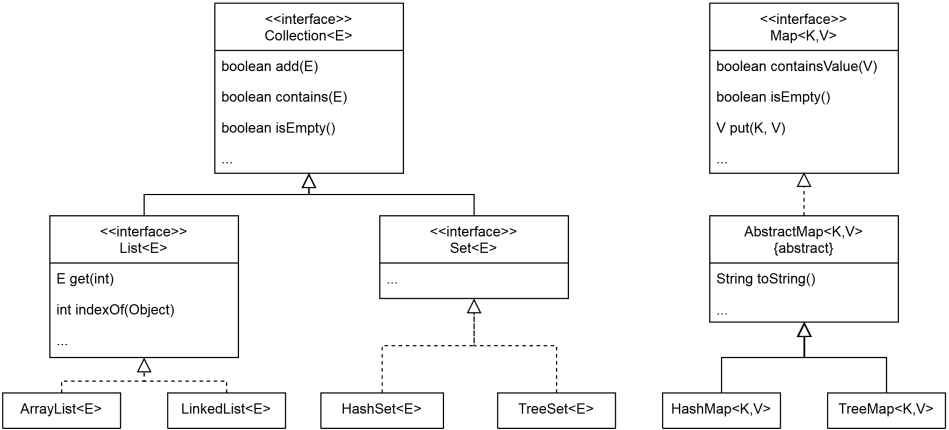
Einfügen am Anfang

```
void insertAtHead(int value) {
    Node newNode = new Node(value);
    newNode.next = head;
    head = newNode;
}
```

Einfügen am Ende

```
void insertAtTail(int value) {
    Node newNode = new Node(value);
    if(head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while(current.next != null) {
        current = current.next;
    }
    current.next = newNode;
}
```

Übersicht des Collection-Frameworks



Gliederung

1 Listen

2 Binäre Suche

3 Sortieralgorithmen

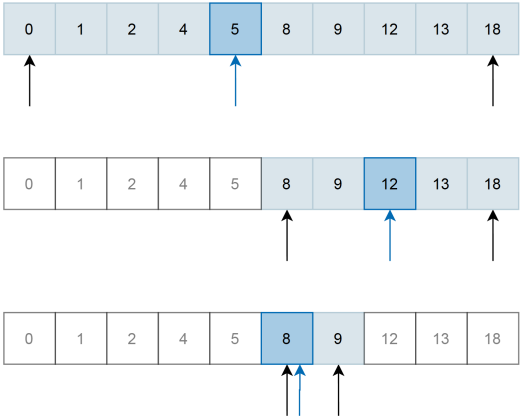
4 Generics

5 Higher Order Functions und Streams

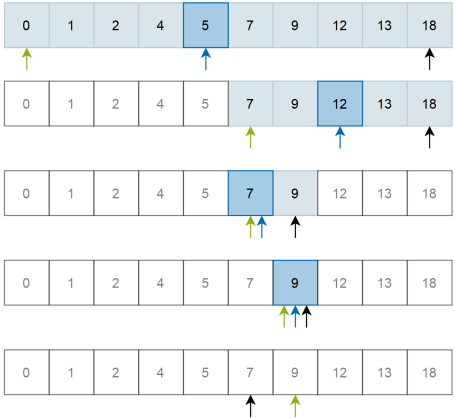
Binäre Suche - Ziel

- Ziel: Finde die Position eines bestimmten Elements in einem sortierten Array.

Beispiel: Suche nach der Zahl 8



Beispiel: erfolglose Suche nach der Zahl 8



Gliederung

1. Listen
2. Binäre Suche
3. Sortieralgorithmen
4. Generics
5. Higher Order Functions und Streams

Sortieralgorithmen

Heinrich Heine Universität Düsseldorf

Um Binäre Suche anwenden zu können muss unser Array jedoch bereits sortiert sein. Dafür benutzen wir Sortieralgorithmen. Welche kennen sie aus der Vorlesung?

Sortieralgorithmen

Heinrich Heine Universität Düsseldorf

Um Binäre Suche anwenden zu können muss unser Array jedoch bereits sortiert sein. Dafür benutzen wir Sortieralgorithmen. Welche kennen sie aus der Vorlesung?

- Insertion Sort
- Merge Sort

Insertion Sort - Funktionsweise

Heinrich Heine Universität Düsseldorf

1. Starte mit dem zweiten Element der Liste.
2. Vergleiche das aktuelle Element mit den Elementen der sortierten Teilliste (links davon).
3. Füge das aktuelle Element an der richtigen Position in die sortierte Teilliste ein.
4. Wiederhole die Schritte 2-3 für alle verbleibenden Elemente.

Insertion Sort - Beispiel

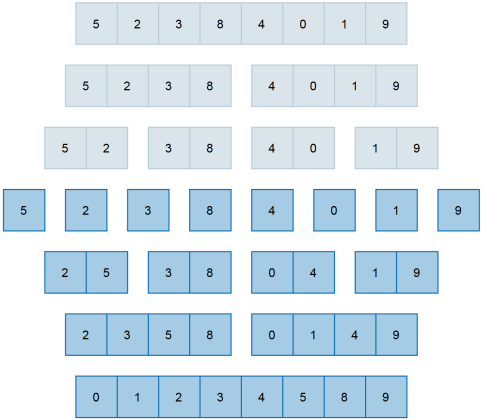
5	2	3	8	0	4
2	5	3	8	0	4
2	3	5	8	0	4
2	3	5	8	0	4
0	2	3	5	8	4
0	2	3	4	5	8

Merge Sort - Funktionsweise

Heinrich Heine Universität Düsseldorf

1. Teile die Liste rekursiv in zwei Hälften, bis jede Teilliste nur noch ein Element enthält.
2. Sortiere die Teillisten, indem du sie paarweise vergleichst und zusammenführst.
3. Wiederhole den Zusammenführungsprozess, bis eine vollständig sortierte Liste entsteht.

Merge Sort - Beispiel



Gliederung

1. Listen
2. Binäre Suche
3. Sortieralgorithmen
4. Generics
5. Higher Order Functions und Streams

Generics

Problem:

- Wenn wir eigene Datenstruktur nun mit neuer eigener Klasse verwenden wollen müssen wir unsere Datenstruktur erneut implementieren.
-

```
    public IntList list = new IntList();
    list.add(new Person("Superman"));          //funktioniert nicht
```

Lösung:

- wir machen aus den festen Datentypen Variablen

Generics

```
public class LinkedList<T>{
    private class Node{
        private T element;
        private Node next;
        private Node(T element, Node next){
            this.element = element;
            this.next = next;
        }
    }
}
```

Wir ändern den festen Datentypen nur auf T. Damit wird dieser variabel. Gleiches machen wir bei allen Methoden

Generics - Verwendung

Unsere Generische Liste können wir nun mit allen Datentypen verwenden, die wir benutzen wollen:

```
LinkedList<String> stringList = new LinkedList<String>();
LinkedList<Person> personList = new LinkedList<Person>();
```

Achtung: primitive Datentypen können wir **nicht** als Generic verwenden:

```
LinkedList<int> intList = new LinkedList<int>();
```

Jedoch existiert die Wrapper Klasse Integer. Folgendes funktioniert also:

```
LinkedList<Integer> intList = new LinkedList<Integer>();
```

Beispielaufgabe

1. Schreiben Sie eine Klasse *Stack*, die einen *int Stack* als Datenstruktur implementiert
2. implementieren Sie *push*
3. implementieren Sie *pop*
4. implementieren Sie *peek*
5. implementieren Sie *isEmpty*
6. implementieren Sie *size*
7. ändern Sie ihre Klasse so, dass diese generisch funktioniert

Gliederung

1. Listen
2. Binäre Suche
3. Sortieralgorithmen
4. Generics
5. Higher Order Functions und Streams

Higher Order Functions

Heinrich Heine Universität Düsseldorf

Higher Order Function (HOF):

- Eine Funktion, die andere Funktionen als Parameter entgegennehmen oder Funktionen zurückgeben
- Beispiele: map,forEach auf Collections
- Datentyp von Funktionen sind funktionale Interfaces
- Funktionale Interfaces: Interfaces mit genau einer abstrakten Methode
- Für Funktionen mit einem Parameter und einem Rückgabewert gibt es das funktionale Interface *Function<E,A>*:

```
interface Function<E,A> {
A run(E input);
}
```

Lambda-Ausdrücke

- Kurzschreibweise einer Funktion, um diese leicht als Parameter zu übergeben
- Syntax: (Parameter) -> {Anweisungen}

```
1    Function<Integer, String> f1 = (Integer n) -> {
2    return "#" + n;
3    };
```

Streams

- Datenstrom
- Lässt sich bei Collections durch Aufruf von *stream()* erzeugen
- verschiedene Operationen auf Streams:
  - *map*: Wendet Funktion auf jedes Element an
  - *forEach*: Beendet Stream
  - *filter*: Filtert Stream nach einer Bedingung
  - *toList*: Erzeugt Liste
- sind idiomatisch außer:
  - Collection wird verändert
  - zu große Labda-Ausdrücke: stattdessen Methode erstellen
  - Seiteneffekte außerhalb von forEach